

Welcome back! Aapka dedication dekh kar lag raha hai ki aap sach mein ek top-tier AI Engineer banne wale hain. Module 1, 2, aur 3 mein humne context ko bada karna, query ko expand karna, aur noise ko compress karna seekha.

Aaj hum padhenge **Module 4: Hybrid Search**. Ye ek aisi technique hai jo 90% production RAG systems (jaise Perplexity ya enterprise bots) use karte hain. Let's dive in!

Module 4: Hybrid Search

1. Why this concept exists

Search duniya mein do alag-alag philosophies hain:

- Keyword Search (Sparse Retrieval):** Exact words match karna (Jaise Google early 2000s mein karta tha).
- Semantic Search (Dense Retrieval):** Meaning match karna (Jaise aajkal vector databases karte hain).

Dono methods ki apni strengths aur weaknesses hain. **Hybrid Search** in dono ko combine karta hai taaki hume "best of both worlds" mil sake — exact keywords bhi match hon aur context/meaning bhi samajh aaye.

2. Problem with traditional RAG (Sparse vs Dense Limitations)

Sirf ek type ki search use karne se kya problem aati hai?

Feature	Sparse Retrieval (BM25)	Dense Retrieval (Vector Search)
What it does	Looks for exact words	Looks for similar meaning
Fails when	User uses synonyms ("laptop" vs "MacBook")	User wants an exact ID/Name ("Error Code 404")
Strengths	Great for Product IDs, Names, Acronyms	Great for concepts, typos, complex questions

The Problem: Agar aap sirf Vector Search (Dense) use kar rahe hain aur user ne search kiya "Policy #HR-9923", toh vector model shayad kisi doosri HR policy ko "semantically similar" maan kar le aaye, aur exact ID miss kar de.

3. Real-world example

Maan lijiye aap ek E-commerce app ka bot bana rahe hain.

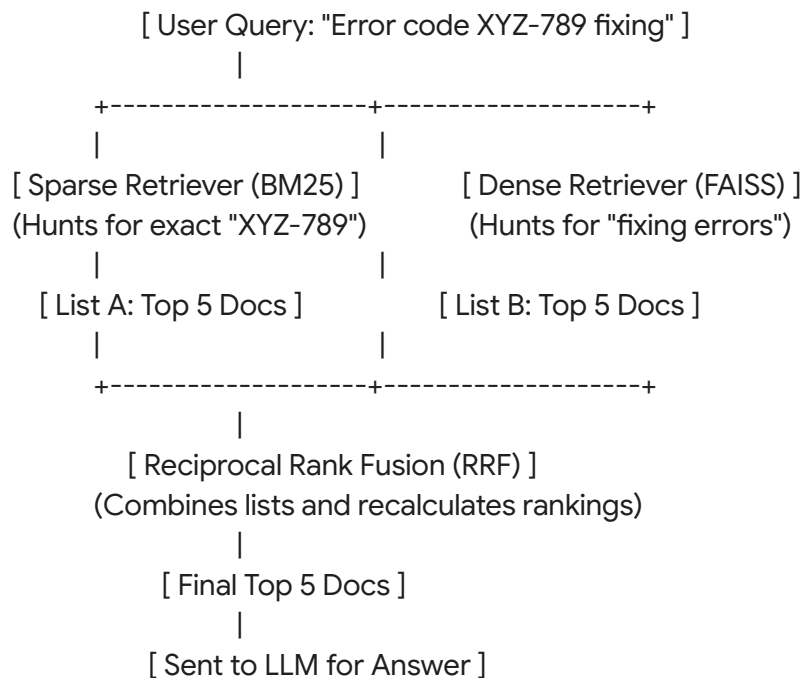
User Query: "Apple iPhone 15 Pro Max 256GB"

- Vector Search (Dense):** Ye samajh jayega ki user ek high-end smartphone dhoondh raha hai. Ye shayad "Samsung S24 Ultra" ya "iPhone 14 Pro" le aaye kyunki unka context similar

hai.

- **BM25 (Sparse):** Ye exact terms "Apple", "iPhone", "15" dhoondhega.
- **Hybrid Search:** Ye ensure karega ki semantically ek phone hi aaye (Vector) lekin specifically "iPhone 15 Pro Max" hi top par aaye (BM25).

4. Visual explanation using diagrams made with text



5. Architecture flow

1. User ka question parallely do alag-alag retrievers ko bheja jata hai.
2. **Dense Retriever** embeddings banata hai aur vector space mein similarity search karke documents laata hai.
3. **Sparse Retriever** (BM25) query ke words count karta hai (TF-IDF formula par based) aur text match karke documents laata hai.
4. Dono retrievers apni-apni ranked lists dete hain.
5. Ek merging algorithm — jise **Reciprocal Rank Fusion (RRF)** kehte hain — dono lists ko combine karta hai. Agar ek document dono lists mein top par hai, toh uska RRF score bohot high ho jayega.
6. Ye combined aur deduplicated list LLM ko bheji jaati hai.

6. Step-by-step implementation

1. Python packages install karein: `pip install rank_bm25 faiss-cpu langchain-community`.
2. Apne documents ko chunks mein split karein.
3. **Sparse Retriever:** LangChain ka BM25Retriever banayein in chunks se.
4. **Dense Retriever:** LangChain ka FAISS vector store banayein aur usko retriever mein convert karein.
5. **Hybrid Retriever:** LangChain ka EnsembleRetriever use karein. Isme dono retrievers ko as a list pass karein, aur weights assign karein (e.g., 50% keyword, 50% vector).
6. Result ko fetch karein.

7. Complete Python code & LangChain implementation

```
import os
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever

# 1. Setup API Keys
os.environ["OPENAI_API_KEY"] = "sk-your-openai-api-key"

# 2. Create Dummy Data for Testing
# Notice how chunk 1 has the exact ID, and chunk 3 has a similar concept
doc_text = """
Chunk 1: To fix the server, check Error Code XYZ-789 in the main terminal.
Chunk 2: The company policy dictates a 30-day notice period.
Chunk 3: System failures and bugs can often be fixed by restarting the router.
Chunk 4: Employee ID XYZ-789 belongs to John Doe from the IT department.
"""

with open("hybrid_test.txt", "w") as f:
    f.write(doc_text)

loader = TextLoader("hybrid_test.txt")
docs = loader.load()
```

```

# Split docs line by line for this example
text_splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=0)
splits = text_splitter.split_documents(docs)

# 3. Setup Dense Retriever (Vector Search)
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(splits, embeddings)
# Fetch top 2 results based on semantic meaning
dense_retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

# 4. Setup Sparse Retriever (Keyword Search)
# Note: You need `pip install rank_bm25` for this to work
sparse_retriever = BM25Retriever.from_documents(splits)
# Fetch top 2 results based on keyword frequency
sparse_retriever.k = 2

# 5. Combine them into an EnsembleRetriever (Hybrid Search!)
# weights=[0.4, 0.6] means 40% importance to BM25, 60% to Vector
hybrid_retriever = EnsembleRetriever(
    retrievers=[sparse_retriever, dense_retriever],
    weights=[0.5, 0.5]
)

# 6. Test the Hybrid System
query = "How to handle Error Code XYZ-789?"
print(f"Query: {query}\n")

# A. Let's see what purely Vector Search gives
print("\n--- DENSE (Vector) RETRIEVER ONLY ---")
for doc in dense_retriever.invoke(query):
    print(doc.page_content.strip())

# B. Let's see what purely Keyword Search gives
print("\n--- SPARSE (BM25) RETRIEVER ONLY ---")
for doc in sparse_retriever.invoke(query):
    print(doc.page_content.strip())

# C. Let's see what Hybrid Search gives!
print("\n--- HYBRID (Ensemble) RETRIEVER ---")
for doc in hybrid_retriever.invoke(query):
    print(doc.page_content.strip())

```

8. Production use case

IT Helpdesk Support Bots: Jab employees issue raise karte hain, unki query mein dono cheezein hoti hain:

1. Natural language (e.g., "Mera laptop screen freeze ho raha hai") - *Vector Search is excellent here.*
 2. Technical specifics (e.g., "ThinkPad T14s v3 BIOS update fail") - *BM25 is excellent here.*
- Hybrid Search ensure karta hai ki model 'freeze' ka meaning bhi samjhe, aur exactly 'ThinkPad T14s' ke hi documents lekar aaye, kisi aur laptop ke nahi.

9. Interview questions

1. **Q: Hybrid Search mein Reciprocal Rank Fusion (RRF) formula kya hota hai?**
A: RRF score calculate karta hai $1 / (k + \text{rank})$. Agar ek doc Vector list mein Rank 1 par hai aur BM25 list mein Rank 2 par hai, toh uska combined score dono ke fractions ko add karke banega. Jo document dono lists mein high hota hai, wo overall top par aa jata hai.
2. **Q: Kya BM25 ko Embeddings ki zaroorat hoti hai?**
A: Nahi. BM25 ek statistical algorithm hai jo Term Frequency-Inverse Document Frequency (TF-IDF) ka advanced version hai. Yeh purely text processing aur word counts par chalta hai, isliye isme vector database ya embedding model ki API cost nahi lagti.
3. **Q: EnsembleRetriever mein weights kaise decide karein?**
A: Ye domain par depend karta hai. Agar system E-commerce (Product search) hai jahan exact terms matter karte hain, toh BM25 ko high weight (e.g., 0.7) de sakte hain. Agar general knowledge/QA bot hai, toh Dense ko high weight (e.g., 0.7) dena chahiye.

10. Advantages

- **Highest Recall:** Ye technical words aur semantic concepts dono ko handle kar leta hai.
- **Out-of-Vocabulary handling:** Vector models naye jargon (like a new internal company acronym) ko samajhne mein struggle karte hain. BM25 exact match se unhe easily dhoondh leta hai.

11. Limitations

- **Double Maintenance:** Ab aapko ek Vector Database (e.g., Pinecone) aur ek Inverted Index (e.g., Elasticsearch for BM25) dono maintain karne padenge.
- **Slower Retrieval:** Dono queries parallel run hoti hain, phir unhe rerank kiya jata hai. Isme slightly zyada computation lagti hai.

12. Best practices

- **Tune the Weights:** Hamesha [0.5, 0.5] par mat ruko. Apne data par test karo. Aksar [0.3 for Sparse, 0.7 for Dense] ek acha starting point hota hai natural language heavy tasks ke liye.
- **Use Proper Databases in Production:** BM25 LangChain mein locally chal raha hai memory ke andar (using rank_bm25). Production mein millions of documents ke liye

aapko **Elasticsearch**, **Milvus**, ya **Weaviate** (jo built-in hybrid search support karte hain) ka use karna chahiye, taaki system scale kar sake.

Assignments & Practical Exercises

- **Exercise 1:** Code mein weights=[0.1, 0.9] (Heavy Vector) aur phir weights=[0.9, 0.1] (Heavy Keyword) set karke dekho aur notice karo ki output ka order kaise change hota hai.
- **Exercise 2:** Ek real PDF load karo (apni pichli exercises wali) aur usme kuch arbitrary IDs insert karo. Phir BM25 aur Vector ko alag-alag search karne ko bolo, aur check karo kaun better ID match karta hai.

Common Mistakes Beginners Make

- *Thinking Hybrid means generating two LLM answers:* Hybrid sirf **retrieval** level par hota hai. Hum documents ko combine karke LLM ko ek hi prompt mein bhejte hain, do alag answers generate nahi karte.
- *Not normalizing scores:* BM25 aur Vector search ke similarity scores bilkul alag scale par hote hain (BM25 can be 15.4, Vector cosine similarity is between 0 to 1). Log directly inke scores ko add karne ki koshish karte hain, jo galat hai. Isiliye hum scores ko nahi, **ranks** ko merge karte hain (RRF). LangChain ka EnsembleRetriever ye natively handle karta hai!

Kaisa laga Hybrid Search ka concept?